LLM Gateway API

FastAPI & LLM Integration

ARQUITETURA ESCALÁVEL E INTEGRAÇÃO DE MODELOS

Hélio Júnior

Visão Geral da Arquitetura

API Layer

Camada de interface e transporte HTTP.

- → Gestão de Rotas
- → Controllers
- → Tratamento de Exceções

Service Layer

Orquestração da lógica de negócio.

- → Regras de Negócio
- → Orquestração de Fluxos
- → Coordenação de Serviços

Client Layer

Abstração de integrações externas.

- → LLM Client (OpenAI/Ollama)
- → Interface Abstrata
- → Baixo Acoplamento

Schemas

Estrutura de dados e validação.

- → Modelos Pydantic
- → Validação de Request
- → Serialização de Response

Stack Tecnológica

Python 3.10+
Linguagem base focada em performance e tipagem.

FastAPI
Framework web moderno e assíncrono.

Pydantic v2
Validação rigorosa e serialização de dados.

OpenAI / HTTPX
Integração robusta com modelos de IA.

Loguru
Logging estruturado e monitoramento eficiente.

Dotenv
Gestão segura de variáveis de ambiente.

Abstração e Integração LLM

Interface Abstrata

Uso da classe base **LLMClient** para garantir desacoplamento total entre a lógica de negócio e os provedores de IA.

Provedores Suportados

- **OpenAI:** Integração nativa com GPT-4 e GPT-3.5.
- **Ollama:** Suporte para modelos locais (Llama, Mistral).
- **Extensível:** Arquitetura pronta para novos providers.

Configuração Dinâmica

Troca de provedor e modelos em tempo de execução via variáveis de ambiente (`.env`), sem alteração de código.

Endpoints e Interface

Health

GET /HEALTH

Monitoramento de integridade.

- Status do sistema
- Versão da API
- Uptime real-time

Prompt

POST /PROMPT

Interface de processamento.

- Envio de prompts
- Controle de temperatura
- Max tokens config

History

GET /HISTORY

Rastreabilidade de dados.

- Logs de requisições
- Respostas geradas
- Metadados do modelo

Docs

UI /DOCS

Interface do desenvolvedor.

- Swagger UI nativo
- ReDoc alternativo
- Testes interativos

Princípios de Engenharia (SOLID)

S

Single Responsibility Principle

Classes com responsabilidades únicas e isoladas (Services, Clients, Routes).

O

Open/Closed Principle

Arquitetura aberta para novos provedores de LLM sem alteração do código core.

L

Liskov Substitution Principle

Substituição transparente entre diferentes implementações de LLMClient.

I

Interface Segregation Principle

Interfaces enxutas e específicas para as necessidades de cada serviço.

D

Dependency Inversion Principle

Dependência de abstrações, garantindo desacoplamento de provedores externos.

Próximos Passos

Persistência SQL
Migração para PostgreSQL com SQLAlchemy para gestão robusta de dados.

Autenticação JWT
Implementação de fluxos OAuth2 para proteção de endpoints e gestão de usuários.

Streaming (SSE)
Suporte a Server-Sent Events para exibição em tempo real das respostas do LLM.

Docker + CI/CD
Containerização completa e pipelines automatizados para deploy contínuo.

Cache / Rate Limiting
Otimização de performance com Redis e controle de tráfego para estabilidade.

Conclusão Estratégica

Escalabilidade

Arquitetura modular preparada para suportar o crescimento de volume e complexidade de requisições.

Sustentabilidade

Código limpo e desacoplado, garantindo baixa manutenção e facilidade de evolução tecnológica.

Prontidão

Solução robusta e documentada, pronta para integração imediata em ecossistemas de produção.